TRANSFORMERS & LLMs

My Complete Study Notebook

Theory · Worked Maths · Diagrams · Interview Prep

1. What a Transformer is
2. Tokens & Embeddings
3. Self-Attention Intuition
4. Queries, Keys, Values
5. Scaled Dot-Product Attention
6. Multi-Head Attention
7. Positional Information
8. Residuals, LayerNorm, FFN
9. The Encoder Stack
10. The Decoder Stack
11. Encoder vs Decoder vs Both
12. Training an LLM
13. Inference & Decoding
14. KV Cache & Fast Attention
15. Vision Transformers
16. Fine-Tuning & LoRA
17. Optimization & Quantization
18. Serving & RAG
19. Interview Revision

Prepared by: Akhil

Akhil

1. What a Transformer Is

The one-line idea

A Transformer is a neural network that reads a whole sequence at once and lets every token **look at every other token** to decide what it means. That "looking" is called **attention**.

- Introduced in 2017 ("Attention Is All You Need", Vaswani et al.).
- Drops recurrence (RNN/LSTM): no step-by-step loop, so training runs in parallel on GPUs.
- Today it powers GPT, Claude, LLaMA, BERT, TS, ViT, Whisper and most multimodal models.

Why it replaced RNNs

Problem with RNNs	How the Transformer fixes it
Processes words one at a time (slow)	All positions computed together
Long-distance info fades (vanishing gradients)	Any token reaches any other in one step
Hard to scale	Quality keeps improving with more data, params, compute (scaling laws)

The big picture



Inside one block



Remember

Attention = tokens talk to each other. FFN = each token thinks on its own. Stack these two ideas N times and you have a Transformer.

Akhil

2. Tokens & Embeddings

Tokenization

Models never see letters. Text is cut into **tokens** (whole words, pieces of words, or bytes), and each token gets an integer ID from a fixed vocabulary.

"Unbelievably fast!" → ["Un", "believ", "ably", " fast", "!"] →

[1844, 29029, 2886, 3049, 0]

(IDs are illustrative; each tokenizer has its own vocabulary.)

Method	Idea	Used in
BPE	Start from characters/bytes, repeatedly merge the most frequent pair	GPT family, LLaMA
WordPiece	Merge pairs that most increase data likelihood; "##" marks word pieces	BERT
Unigram / SentencePiece	Start big, prune tokens; works on raw text incl. spaces	T5, many multilingual models

Rule of thumb: 1 English token ≈ ¾ of a word ≈ 4 characters.

Embeddings

- An **embedding table** of shape $(V \times d_{model})$ turns each ID into a learned vector.
- Similar meanings end up close together: king − man + woman ≈ queen.
- Example sizes: $V \approx 32k–200k$, $d_{model} = 768$ (small) to 8192+ (large).

ID 3049 (" fast") → row 3049 of table → $\begin{pmatrix} 0.21 & -0.47 & 0.08 & \dots & 0.33 \end{pmatrix}$

$(1 \times d_{model})$

Remember

Input to the first block = token embedding + position information (page 8). Output head often reuses the same embedding table ("weight tying").

Akhil

3. Self-Attention: the Intuition

The problem

In "I sat by the **bank** of the river", the word **bank** only makes sense after looking at **river**. A word's meaning depends on its neighbours, near or far.

The library analogy

- **Query**: the question I'm asking ("what kind of bank am I?")
- **Key**: the label on every other word ("I'm about water")
- **Value**: the content a word hands over if chosen

Process

1. Compare my query with every key.
2. Turn the match scores into weights that add to 1.
3. Take the weighted mix of values.

Picture it

"bank" attends to →	I	sat	by	the	bank	of	river
weight (example)	0.02	0.08	0.05	0.03	0.22	0.04	**0.56**

The new vector for "bank" is mostly built from "river", so it now means river bank, not money bank.

Key properties

- **Global:** any token can reach any other in one hop.
- **Dynamic:** weights are computed from the input, so they differ for every sentence.
- **Parallel:** it's all matrix multiplication, so it suits GPUs well.
- **Cost:** n tokens → $n \times n$ scores, so $O(n^2)$ time and memory.

Remember

"Self"-attention = the queries, keys and values all come from the same sequence. Cross-attention = queries from one sequence, keys/values from another.

Akhil

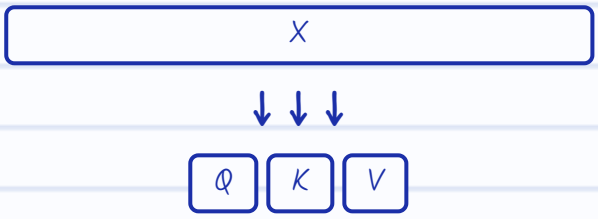
4. Queries, Keys & Values

Three learned projections

Each token vector x is multiplied by three trainable matrices. Same input, three different "views":

$$Q = X \cdot W_Q \qquad K = X \cdot W_K \qquad V = X \cdot W_V$$

Matrix	Shape
X (inputs)	$n \times d_{model}$
W_Q, W_K	$d_{model} \times d_k$
W_V	$d_{model} \times d_v$
Q, K	$n \times d_k$
V	$n \times d_v$



Mini example (2 tokens, $d_{model} = 3$, $d_k = 2$)

$$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \end{pmatrix} \quad W_Q = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix} \Rightarrow Q = \begin{pmatrix} 1 & 2 \\ 1 & 2 \end{pmatrix}$$

Row 1: $[1,2,0] \cdot W_Q = [1 \cdot 1 + 2 \cdot 0 + 0 \cdot 1,\ 1 \cdot 0 + 2 \cdot 1 + 0 \cdot 1] = [1, 2]$. Row 2: $[0,1,1] \cdot W_Q = [0+0+1,\ 0+1+1] = [1, 2]$.

K and V are computed the same way with their own matrices.

Why three separate matrices?

- "What I'm looking for" (Q) differs from "how I can be found" (K).
- "What I advertise" (K) differs from "what I actually pass on" (V).
- Separate weights let the model learn each role independently.

Remember

Q·K decides where to look. V decides what gets copied.

Akhil

5. Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}} + M\right) \cdot V$$

M = optional mask (0 = allowed, $-\infty$ = blocked)

Five steps

Scores $Q K^T$ → Scale $\div \sqrt{d_k}$ → Mask (if any) → Softmax per row → $\times V$ weighted sum

Why divide by $\sqrt{d_k}$?

A dot product of two random d_k-dim vectors has variance $\approx d_k$. Big scores push softmax into a near one-hot shape where gradients vanish. Dividing by $\sqrt{d_k}$ brings the variance back to ≈ 1.

Worked example (2 tokens, $d_k = 2$)

$$Q = \begin{pmatrix} 2 & 0 \\ 1 & 1 \end{pmatrix} \quad K = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix} \quad V = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

① $Q K^T = \begin{pmatrix} 2 & 0 \\ 2 & 2 \end{pmatrix}$ ② $\div \sqrt{2} \approx 1.41 \rightarrow \begin{pmatrix} 1.41 & 0 \\ 1.41 & 1.41 \end{pmatrix}$

③ softmax row 1: $e^{1.41} = 4.10$, $e^{0} = 1 \rightarrow 4.10/5.10 = 0.80$, $1/5.10 = 0.20$

$$\text{weights} = \begin{pmatrix} 0.80 & 0.20 \\ 0.50 & 0.50 \end{pmatrix}$$ ④ $\times V \Rightarrow \text{output} = \begin{pmatrix} 0.80 & 0.20 \\ 0.50 & 0.50 \end{pmatrix}$

Token 1's query matches key 1 strongly, so 80% of its output comes from value 1. Token 2 matches both keys equally, so it gets a 50/50 blend.

Remember

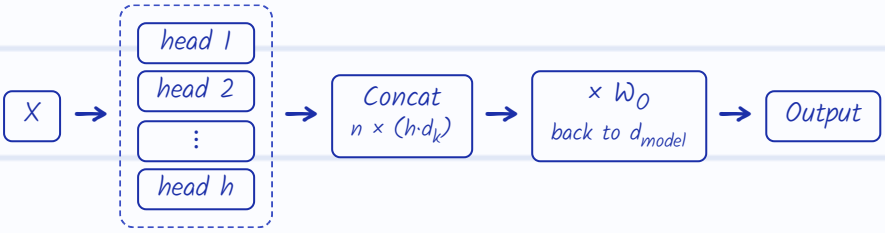
Each output row is a weighted average of value rows, and each weight row sums to 1.

Akhil

6. Multi-Head Attention

Idea

*One attention pattern can't capture everything. Run **h** smaller attentions in parallel, each with its own W_Q, W_K, W_V, then merge the results.*



$$head_i = Attention(X\,W_Q^i,\ X\,W_K^i,\ X\,W_V^i) \qquad MHA(X) = [head_1\ ;\ \dots\ ;\ head_h]\,W_O$$

What different heads learn (observed in real models)

- *Previous-token heads, which look one step back.*
- *Syntax heads, e.g. verb → its subject.*
- *Coreference heads, e.g. "she" → the person named earlier.*
- *Induction heads, which copy patterns seen earlier in the context and drive in-context learning.*

Real dimensions: LLaMA-2 7B

- $d_{model} = 4096$
- heads $h = 32$
- $d_k = 4096 / 32 = 128$ per head
- layers = 32

Cost check

Total compute ≈ one big head of size d_{model}, because the heads split the width instead of multiplying it.

Variants that save memory

Type	KV heads	Trade-off
MHA (multi-head)	h	Best quality, biggest KV cache
GQA (grouped-query)	g (e.g. 8)	Groups of query heads share K/V. Used in LLaMA-2 70B, LLaMA-3, Mistral
MQA (multi-query)	1	Smallest cache, small quality drop

Akhil

7. Positional Information

Why it's needed

Attention is **permutation-invariant**: shuffle the tokens and the outputs shuffle the same way. Without position info, "dog bites man" = "man bites dog".

Option A: Sinusoidal (original paper)

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d}) \quad PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$$

pos	dim 0 (sin)	dim 1 (cos)
0	0.00	1.00
1	0.84	0.54
2	0.91	−0.42
3	0.14	−0.99

- Low dims spin fast and high dims spin slowly, like the hands of a clock.
- No parameters to learn.
- Added to the token embedding.

Option B: Learned absolute

A trainable table with one vector per position (BERT, GPT-2). Simple, but it can't go past the max length seen in training.

Option C: RoPE (rotary), today's default

- Rotate each (q, k) pair of dims by an angle proportional to position.
- The dot product q·k then depends only on the **relative distance** (m − n).
- Applied inside attention, not added to embeddings.
- Used in LLaMA, Mistral, Qwen and others. Context can be stretched with scaling tricks (NTK, YaRN).

Option D: ALiBi

No embedding at all: subtract a penalty ∝ distance from each attention score. Extrapolates well to longer inputs.

Remember

Absolute = "I am at slot 7". Relative/RoPE = "you are 3 tokens behind me". Relative schemes generalize better to long contexts.

Akhil

8. Residuals, LayerNorm & FFN

Residual (skip) connections

$$x_{out} = x + Sublayer(x)$$

- Gradients flow straight through the "+", which makes training 100+ layers possible.
- Each layer only learns a correction to its input (the "residual stream").

Layer Normalization

$$LN(x) = \gamma \cdot \frac{x - \mu}{\sqrt{(\sigma^2 + \varepsilon)}} + \beta$$

- Normalizes across the features of **one token** (not across the batch like BatchNorm).
- **RMSNorm** skips the mean and scales by the root mean square only. It's cheaper and used in LLaMA.

	Post-LN (2017 original)	Pre-LN (modern LLMs)
Formula	$LN(x + Sub(x))$	$x + Sub(LN(x))$
Training	Needs careful warmup	More stable at depth

Position-wise Feed-Forward Network

$$FFN(x) = W_2 \cdot \phi(W_1 x + b_1) + b_2$$

x (d_{model}) → expand ($d_{ff} \approx 4 \cdot d_{model}$) → activation ϕ → shrink (d_{model})

- ϕ = ReLU (original), GELU (BERT, GPT), **SwiGLU** (LLaMA, PaLM).
- Applied to each token separately with the same weights.
- Holds about ⅔ of all parameters and acts as the model's "memory" of facts.

Remember

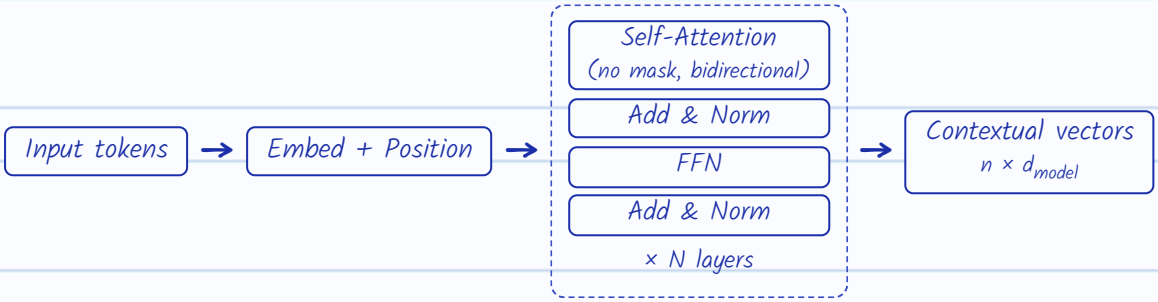
Block = $x + Attn(LN(x))$, then $x + FFN(LN(x))$. That's the whole pre-LN block.

Akhil

9. The Encoder Stack

Job

Turn the input into **contextual vectors**: one rich vector per token that knows about the whole sentence, left and right.



Key facts

- **Bidirectional:** every token sees every other token, with no mask.
- Input and output lengths are the same: n vectors in, n vectors out.
- A special **[CLS]** token's output is often used as a summary of the whole input.

How encoders are trained

Objective	What happens
Masked LM (MLM)	Hide ~15% of tokens and predict them from both sides. "The [MASK] barked" → "dog"
Next sentence prediction	Does sentence B follow A? (BERT. Later dropped by RoBERTa)
Replaced token detection	Spot which tokens a small generator swapped (ELECTRA)

Good at

- Classification: sentiment, spam, intent.
- Token labelling: named entities, POS tags.
- Embeddings for search and RAG (sentence-transformers).
- Extractive QA: find the answer span.

Remember

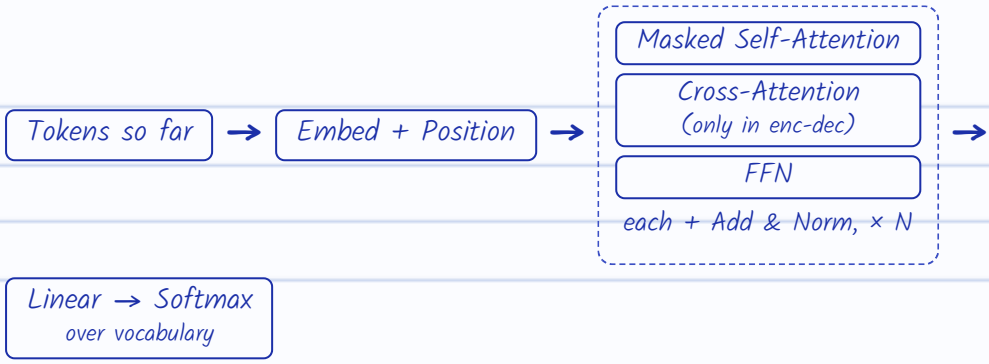
Encoders *understand* but don't *generate*. Examples: BERT, RoBERTa, DeBERTa, ES / BGE embedding models.

Akhil

10. The Decoder Stack

Job

Generate output **one token at a time**, each new token conditioned on everything before it.



Causal mask

Token t may look at positions ≤ t only.
Future positions get −∞ before the softmax, so their weight becomes 0.
This lets training check **every position in parallel** without cheating.

✓	X	X	X
✓	✓	X	X
✓	✓	✓	X
✓	✓	✓	✓

rows = query position, cols = key position

Cross-attention (encoder–decoder models)

- **Q** comes from the decoder: "what do I need next?"
- **K, V** come from the encoder output: "here's the source sentence".
- Translation example: while writing "Hund", the decoder attends to "dog" in the English input.

Teacher forcing

During training the decoder is fed the **true** previous tokens (shifted right), not its own guesses. It's fast and stable. The gap between training and generation is called *exposure bias*.

Remember

Decoder = masked self-attention (+ cross-attention if there's an encoder) + FFN.
GPT-style LLMs are a decoder alone.

Akhil

II. Encoder vs Decoder vs Both

	Encoder-only	Decoder-only	Encoder–Decoder
Attention	Bidirectional	Causal (left → right)	Bi in encoder, causal + cross in decoder
Pre-training	Masked LM	Next-token prediction	Span corruption / denoising
Strength	Understanding, embeddings	Open-ended generation, chat, code	Input → output transforms
Examples	BERT, RoBERTa, DeBERTa	GPT, Claude, LLaMA, Mistral	T5, BART, original Transformer, Whisper
Typical tasks	Classify, NER, retrieval	Chatbots, agents, reasoning	Translation, summarization, speech-to-text

Why decoder-only won for LLMs

- One simple objective (predict the next token) works on **any** text, so the data supply is effectively unlimited.
- Every task can be phrased as "continue this text", which is why prompting works.
- Scales cleanly and supports the KV cache for fast generation.
- In-context learning shows up with scale: the model learns from examples placed in the prompt.

How to choose

Need a label or vector? → Encoder

Need free-form text? → Decoder

Fixed input → different output (and want a small model) → Encoder–Decoder

Prefix-LM (a hybrid)

A decoder that lets the prompt part attend bidirectionally and the generated part causally (used in some UL2 / GLM setups).

Remember

The architecture is the same block every time. The **mask** and the **training objective** are what make it an encoder or a decoder.

Akhil

12. Training an LLM

The three stages

Pre-training *trillions of tokens, next-token* → Supervised fine-tuning *instruction → answer pairs* → Preference tuning *RLHF / DPO / RLAIF*

Objective: next-token prediction

$$\mathcal{L} = -\frac{1}{T}\sum_t \log P_\theta(x_t \mid x_{<t})$$

Cross-entropy between the predicted distribution and the true next token. One sequence of T tokens gives T training signals at once.

Perplexity = $e^{\mathcal{L}}$, *"how many choices the model is torn between". Lower is better.*

Training loop

Batch of token IDs → *Forward pass* → *Loss* → *Backprop* → *AdamW step* ↺

Typical recipe

- *Optimizer: AdamW (β_1 0.9, β_2 0.95)*
- *LR: warmup → cosine decay, peak ~3e-4 (smaller for big models)*
- *Weight decay 0.1, gradient clip 1.0*
- *Precision: BF16 mixed*
- *Batch: millions of tokens per step*
- *Context: 2k–8k in pre-training, extended later*
- *Data: web, books, code, papers, filtered and de-duplicated*

Scaling laws (Chinchilla rule)

For a fixed compute budget, train with roughly ***20 tokens per parameter****. A 7B model → ~140B tokens is "compute-optimal". Modern models train far longer (LLaMA-3 8B used 15T tokens) so they're cheaper to run.*

Alignment in one line each

- ***SFT:*** *imitate good answers.*
- ***RLHF:*** *train a reward model on human preferences, then optimize the policy with PPO.*
- ***DPO:*** *skip the reward model and learn directly from (chosen, rejected) pairs.*

Akhil

13. Inference & Decoding

The generation loop

Prompt "The monsoon rain" → Model → Logits one per vocab token → Pick a token " fell" → Append & repeat

Stops at an end-of-sequence token, a stop string, or max_tokens.

Logits → probabilities (with temperature T)

$$P(i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

T	Effect	Use for
→ 0	Always the top token (greedy)	Extraction, maths, code
0.7–1.0	Balanced	Chat, general writing
> 1	Flatter, more random	Brainstorming (risky)

Decoding strategies

Strategy	How	Pro / Con
Greedy	argmax every step	Fast, deterministic / repetitive
Beam search	Keep B best partial sequences	Good for translation / bland for chat
Top-k	Sample from the k most likely tokens	Simple / fixed k ignores confidence
Top-p (nucleus)	Sample from the smallest set with $\Sigma p \geq p$	Adapts to confidence / one more knob
Min-p	Keep tokens with $p \geq min_p \times p_{max}$	Stable at high T

Other knobs

- Repetition / frequency penalty: lowers the logits of tokens already used.
- Structured output: constrain sampling to valid JSON with a grammar.

Remember

Prefill (reading the prompt) is parallel and compute-bound. Decode (one token at a time) is sequential and memory-bound.

Akhil

14. KV Cache & Fast Attention

The waste without a cache

At step t the model needs K and V for all earlier tokens. Recomputing them every step repeats work: generating n tokens costs $O(n^2)$ extra projections.

The fix

New token t → Compute q_t, k_t, v_t only → Append k_t, v_t to cache →
Attend q_t over whole cache → Next token

Old K, V never change (causal mask), so storing them is safe. Each step's attention costs $O(t)$ instead of recomputing $O(t^2)$.

How big is the cache?

$$\text{bytes / token} = 2 \times \text{layers} \times \text{kv_heads} \times d_{\text{head}} \times \text{bytes_per_value}$$

LLaMA-2 7B, FP16

$2 \times 32 \times 32 \times 128 \times 2\ B = 524{,}288\ B \approx$ **0.5 MB per token**

4,096-token context ≈ **2 GB** per sequence. That's why long contexts and big batches eat GPU memory.

Ways to shrink or speed it up

Technique	What it does
GQA / MQA	Fewer K/V heads, so a 4–32× smaller cache
PagedAttention (vLLM)	Cache stored in pages like OS memory, so no fragmentation
KV quantization	Store the cache in FP8 / INT8
Sliding-window attention	Only attend to the last W tokens (Mistral)
FlashAttention	Computes exact attention in on-chip SRAM tiles without writing the n×n matrix, so it's much faster with $O(n)$ memory
Prefix caching	Reuse the cache for a shared system prompt across requests

Akhil

15. Vision Transformers (ViT)

"An image is worth 16×16 words"

Image $224 \times 224 \times 3$ → Cut into patches 16×16 → Flatten + Linear each → d_{model} → + [CLS] + positions → Encoder × L → [CLS] → class

Patch arithmetic

- Patches: $(224/16)^2 = 14^2 =$ **196 tokens** (+1 [CLS] = 197).
- One patch = 16 × 16 × 3 = **768 numbers**, projected to d_{model}.
- Halving the patch size to 8 → 784 tokens → ~16× attention cost.

ViT vs CNN

	CNN	ViT
Built-in bias	Locality, translation invariance	Almost none; learns it from data
Small data	Wins	Needs big pre-training or tricks
Large data	Plateaus	Keeps scaling
Receptive field	Grows layer by layer	Global from layer 1

Family tree

Model	Key idea
DeiT	Data-efficient training with a distillation token
Swin	Local shifted windows + hierarchy, good for detection
MAE	Mask 75% of patches, reconstruct them (self-supervised)
DINOv2	Self-distillation, strong general image features
CLIP	Image encoder + text encoder trained to match captions
LLaVA-style VLMs	ViT features → projector → LLM tokens

Remember

Same Transformer, new tokenizer: patches instead of words. Audio (Whisper) uses spectrogram frames the same way.

Akhil

16. Fine-Tuning & LoRA

Options from heavy to light

Method	Trainable	When
Full fine-tuning	100%	Lots of data and GPUs, big domain shift
LoRA	~0.1–1%	Default choice for adapting LLMs
QLoRA	LoRA on a 4-bit frozen base	Fine-tune a 65B model on one 48 GB GPU
Adapters	Small bottleneck layers	Many tasks, swap modules
Prompt / prefix tuning	A few virtual tokens	Very light tasks
Prompting / few-shot	0%	Try this first

LoRA: Low-Rank Adaptation

$$W' = W_{frozen} + \Delta W, \quad \Delta W = \frac{\alpha}{r} \cdot B \cdot A$$

- W: $d \times k$, frozen
- B: $d \times r$, starts at 0
- A: $r \times k$, random init
- $r \ll d$ (e.g. 8, 16, 64)
- Usually applied to W_Q, W_V (or all linear layers)

Parameter count

One 4096 × 4096 matrix = 16.8 M params.

LoRA $r = 8$: $8 \times (4096 + 4096) = 65{,}536$

$\Rightarrow$ **0.39%** of the original.

B starts at 0, so $\Delta W = 0$ at step 0 and the model starts exactly as the base. After training, ΔW can be **merged** into W, so inference costs nothing extra.

Adapter block



Fine-tuning checklist

- Clean, consistent data beats more data. A few thousand good examples are often enough.
- Use the base model's chat template exactly.
- Hold out an eval set and watch for overfitting and forgetting.
- Start with LR ~1e-4 to 2e-4 for LoRA and 1–3 epochs.

Akhil

17. Optimization & Quantization

Training at scale

Technique	Idea
Mixed precision (BF16)	Compute in 16-bit with FP32 master weights. ~2× faster, half the memory
Gradient checkpointing	Drop activations and recompute them in the backward pass, trading compute for memory
Gradient accumulation	Sum several micro-batches before one optimizer step
Data parallel	Copy the model on each GPU, split the batch, all-reduce the gradients
ZeRO / FSDP	Shard optimizer states, gradients and weights across GPUs
Tensor / pipeline parallel	Split each layer's matrices / split the layers across GPUs

Memory math (weights only)

$$\text{memory} \approx \text{parameters} \times \text{bytes per parameter}$$

7B model in...	Bytes / param	Weights
FP32	4	28 GB
FP16 / BF16	2	14 GB
INT8	1	7 GB
INT4	0.5	3.5 GB

Full training with AdamW needs ~16 bytes/param (weights + grads + 2 optimizer moments + FP32 copy), so 7B → ~112 GB before activations.

Quantization methods

- **PTQ** (post-training): GPTQ, AWQ, and GGUF for llama.cpp. Minutes to apply, small accuracy loss at 4-bit.
- **QAT** (quantization-aware training): simulate low precision during training for better accuracy.
- Other tricks: pruning, knowledge distillation (a small student copies a big teacher), speculative decoding.

Known limitations

- $O(n^2)$ attention cost, so long context is expensive.
- Hallucinations: fluent but wrong answers.
- Knowledge cutoff, and weak exact arithmetic without tools.
- Bias from the training data, and prompt-injection risk in apps.

Akhil

18. Serving & RAG

Serving stack

App / user → API gateway (auth, rate limits) → Inference server (vLLM · TGI · TensorRT-LLM · llama.cpp) → GPU(s)

- **Continuous batching:** new requests join the running batch at every step, which greatly raises throughput.
- **Metrics:** TTFT (time to first token), TPOT (time per output token), tokens/sec, cost per 1M tokens.
- **Streaming:** send tokens as they're generated, so the response feels faster.

RAG: Retrieval-Augmented Generation

Give the model the right facts at question time instead of hoping it memorized them.

Indexing (offline)

Docs → Chunk (300–800 tokens, overlap) → Embed → Vector DB + metadata

Answering (online)

Question → Embed → Top-k search (+ keyword / hybrid) → Re-rank →

Prompt = context + question → LLM answer with citations

Choices to make

- Chunking: fixed, recursive, by heading, semantic
- Embeddings: e5, bge, OpenAI / Cohere / Voyage
- Stores: FAISS, pgvector, Qdrant, Pinecone, Weaviate, Chroma
- Similarity: cosine / dot product

Evaluate it

- Retrieval: recall@k, MRR
- Faithfulness: is the answer supported by the context?
- Answer relevance and correctness
- Latency and cost per query

RAG or fine-tune?

RAG for *knowledge* that changes or must be cited. Fine-tuning for *behaviour*: style, format, domain language. They combine well.

Akhil

19. Interview Revision

Quick-fire Q & A

1. **Why scale by $\sqrt{d_k}$?**
   Keeps the dot products' variance near 1 so softmax doesn't saturate and gradients don't vanish.
2. **Complexity of self-attention?**
   $O(n^2 \cdot d)$ time and $O(n^2)$ memory for the score matrix. FlashAttention removes the memory part.
3. **Why do we need positional encoding?**
   Attention ignores order. Position info lets the model tell "dog bites man" from "man bites dog".
4. **BERT vs GPT?**
   BERT: encoder, bidirectional, masked LM, built for understanding. GPT: decoder, causal, next-token, built for generation.
5. **What does the causal mask do?**
   Sets future scores to $-\infty$, so each token only sees the past. That allows parallel training without leaking answers.
6. **Why multiple heads?**
   Each head learns a different relationship (syntax, coreference, position) in its own subspace.
7. **What is the KV cache?**
   Stored keys and values of past tokens, so each new token only computes its own. It trades memory for speed.
8. **LoRA in one sentence?**
   Freeze W and learn a low-rank update BA, training under 1% of the parameters, then merge it for free inference.
9. **Temperature vs top-p?**
   Temperature reshapes the whole distribution. Top-p cuts off the unlikely tail before sampling.
10. **How do you reduce hallucinations?**
    RAG with citations, lower temperature, ask the model to say "I don't know", verification steps, and evals.
11. **Pre-LN vs Post-LN?**
    Pre-LN normalizes before each sublayer, which makes deep models train more stably. Modern LLMs use it.
12. **Why decoder-only for LLMs?**
    One universal objective, unlimited text, works with prompting and in-context learning, and generates efficiently with a KV cache.

Revision checklist

- ☐ Write the attention formula from memory
- ☐ Trace shapes through one block
- ☐ Explain RoPE vs sinusoidal
- ☐ Compute KV-cache size for any model
- ☐ LoRA parameter count for any r
- ☐ Model memory at FP16 / INT4
- ☐ Draw the RAG pipeline
- ☐ Compare decoding strategies

Final word

Attention to understand, FFN to think, residuals to stay stable. Keep learning, keep building!

Akhil